DoSLR Puducherry WebGIS — System & Database Diagrams

RFP No: M-13/1237/2025 — Module 2 (WebGIS) + Module 3 (Mobile / Georeferencing)

A complete visual model of the workload: how the pieces connect, how data moves, how each process runs step-by-step, and how the database is wired together.

How to read these diagrams

Data-flow / context notation used below:

- `(( double-circle ))` = the system as a whole, or a process
- `[ rectangle ]` = an external entity (a person or an outside system)
- `[( cylinder )]` = a data store (a database / table group)
- arrows = direction data travels, labelled with *what* travels

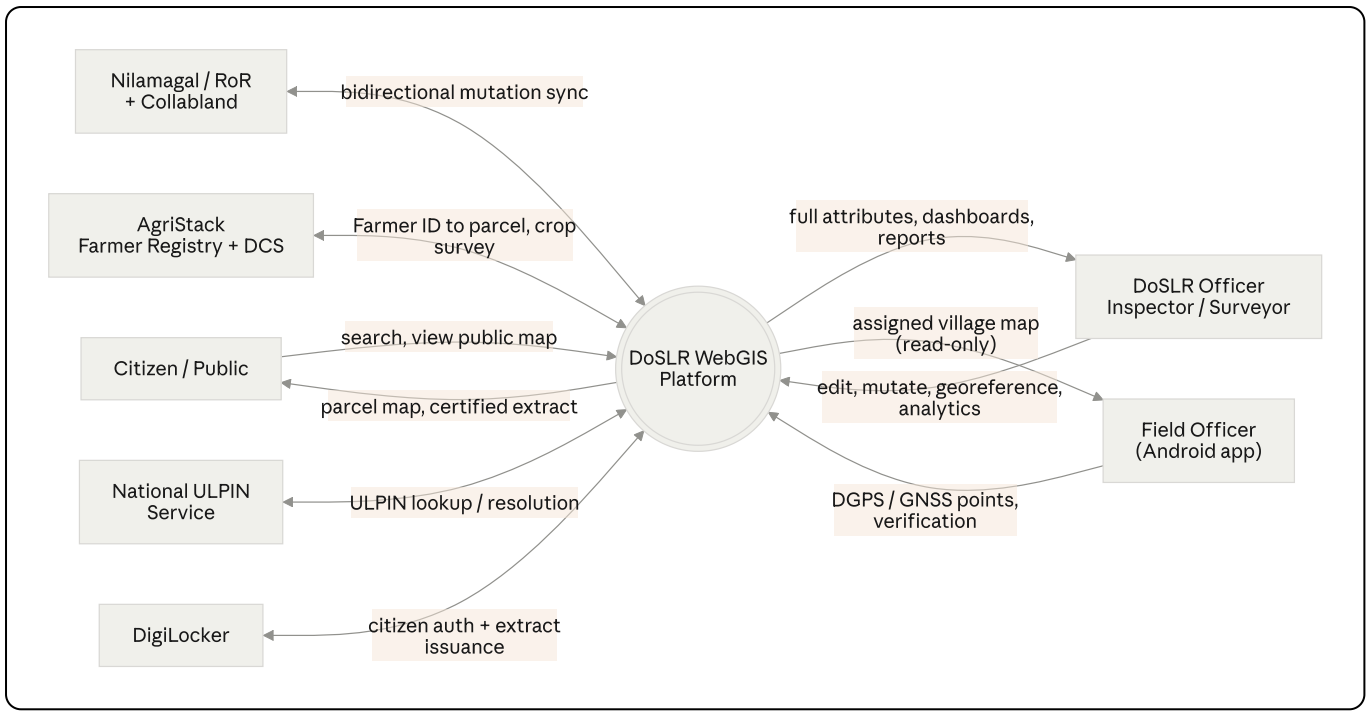
Table of contents

- Part A — System views (context, architecture)
- Part B — Database ERD (schema map + detailed entity diagrams)
- Part C — Data-flow diagrams
- Part D — Process flowcharts (the actual step-by-step workflows)
- Part E — State & lifecycle diagrams
- Part F — Sequence diagrams (end-to-end interactions)

Part A — System Views

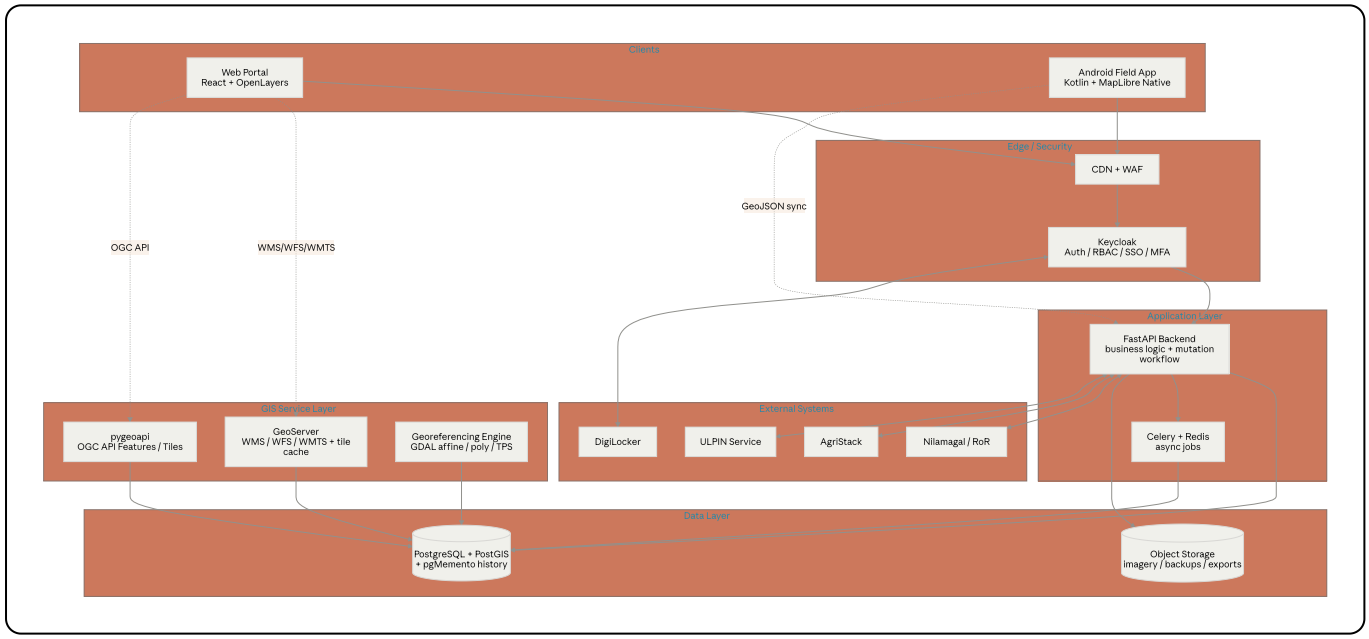
A.1 System Context (Level-0 Data Flow)

Who and what touches the platform, and what crosses each boundary.



A.2 Container / Component Architecture

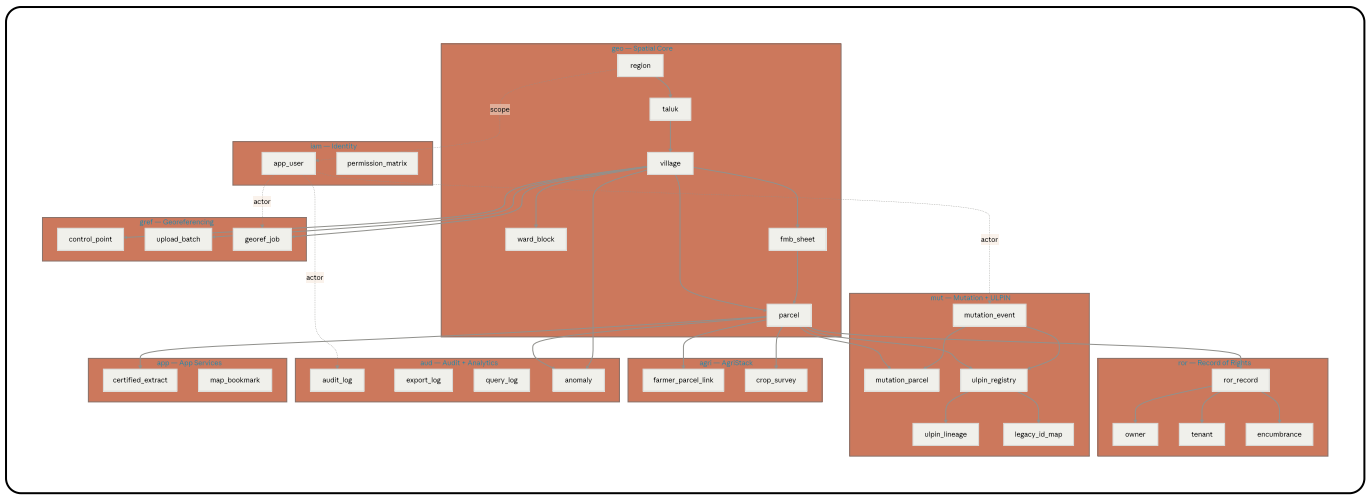
The runnable pieces and which open-source tool plays each role.



Part B — Database ERD

B.1 Schema Map (all 28 tables, grouped by schema, with foreign-key links)

The big picture: every table, which schema it lives in, and how the keys connect across schemas. `parcel` is the spine that almost everything hangs off.



B.2 Detailed ERD — Spatial Core + Record of Rights

The cadastral side (`geo`) and the RoR side (`ror`) are stored separately on purpose so their values can be compared for variance analytics.

```
erDiagram
    REGION ||--o{ TALUK : contains
    TALUK ||--o{ VILLAGE : contains
    VILLAGE ||--o{ WARD_BLOCK : contains
    VILLAGE ||--o{ FMB_SHEET : holds
    VILLAGE ||--o{ PARCEL : holds
    FMB_SHEET ||--o{ PARCEL : "lineage source"
    PARCEL ||--o{ ROR_RECORD : "described by"
    ROR_RECORD ||--o{ OWNER : "owned by"
    ROR_RECORD ||--o{ TENANT : "tenanted by"
    ROR_RECORD ||--o{ ENCUMBRANCE : "encumbered by"

    REGION {
        int region_id PK
        varchar code
        varchar name
        geometry geom
    }
    TALUK {
        int taluk_id PK
        int region_id FK
        varchar name
        geometry geom
    }
    VILLAGE {
        int village_id PK
        int taluk_id FK
        varchar name
        geometry geom
    }
    WARD_BLOCK {
        int ward_id PK
        int village_id FK
        varchar kind
        geometry geom
    }
    FMB_SHEET {
        int fmb_sheet_id PK
        int village_id FK
        varchar sheet_no
        varchar legibility
        int survey_year
        geometry outline_geom
    }
    PARCEL {
        bigint parcel_id PK
        char ulpin UK
        varchar survey_no
        varchar sub_div_no
        int village_id FK
        int fmb_sheet_id FK
        enum classification
```

```
        geometry geom
        numeric computed_area_sqm
        enum status
        timestamptz valid_from
        timestamptz valid_to
    }
    ROR_RECORD {
        bigint ror_record_id PK
        bigint parcel_id FK
        varchar survey_no
        varchar sub_div_no
        enum classification
        numeric ror_extent_sqm
        numeric kist_amount
        varchar patta_no
        varchar source_system
        varchar ror_txn_id
    }
    OWNER {
        bigint owner_id PK
        bigint ror_record_id FK
        varchar owner_name
        varchar owner_ref_key
        numeric share_fraction
        varchar share_text
    }
    TENANT {
        bigint tenant_id PK
        bigint ror_record_id FK
        varchar tenant_name
        numeric share_fraction
    }
    ENCUMBRANCE {
        bigint encumbrance_id PK
        bigint ror_record_id FK
        varchar enc_type
        numeric amount
        date registered_date
        boolean is_active
    }
```

B.3 Detailed ERD — Mutation, Workflow & ULPIN Lineage

How a mutation event ties parent and child parcels together, drives the approval workflow, and re-issues ULPINs with full lineage.

```
erDiagram
    MUTATION_EVENT ||--o{ MUTATION_PARCEL : affects
    PARCEL ||--o{ MUTATION_PARCEL : "involved in"
    PARCEL ||--o{ ULPIN_REGISTRY : "identified by"
    MUTATION_EVENT ||--o{ ULPIN_REGISTRY : "issues / retires"
    ULPIN_REGISTRY ||--o{ ULPIN_LINEAGE : "parent of"
    ULPIN_REGISTRY ||--o{ LEGACY_ID_MAP : "cross-refs"

    MUTATION_EVENT {
        bigint mutation_id PK
        enum mutation_type
        enum status
        varchar ror_txn_id
        geometry before_geom
        geometry after_geom
        jsonb before_attrs
        jsonb after_attrs
        uuid initiated_by
        uuid approved_by
        timestamptz approved_at
        timestamptz synced_at
    }
    MUTATION_PARCEL {
        bigint id PK
        bigint mutation_id FK
        bigint parcel_id FK
        enum role
    }
    PARCEL {
        bigint parcel_id PK
        char ulpin UK
        varchar survey_no
        enum status
    }
    ULPIN_REGISTRY {
        char ulpin PK
        bigint parcel_id FK
        enum status
        timestamptz issued_at
        timestamptz retired_at
    }
    ULPIN_LINEAGE {
        bigint id PK
        char parent_ulpin FK
        char child_ulpin FK
        enum relation
        bigint mutation_id FK
    }
    LEGACY_ID_MAP {
        bigint id PK
        char ulpin FK
        varchar legacy_system
```

```
        varchar legacy_id
    }
```

B.4 Detailed ERD — Georeferencing, AgriStack, Identity, Audit

The Module-3 georeferencing tables plus the supporting integration, identity, and audit tables.

```
erDiagram
    VILLAGE ||--o{ CONTROL_POINT : "located in"
    VILLAGE ||--o{ GEOREF_JOB : "scoped to"
    VILLAGE ||--o{ UPLOAD_BATCH : "for"
    PARCEL ||--o{ FARMER_PARCEL_LINK : "cultivated by"
    PARCEL ||--o{ CROP_SURVEY : "crop on"
    PARCEL ||--o{ CERTIFIED_EXTRACT : "extract of"
    PARCEL ||--o{ ANOMALY : "flagged in"
    APP_USER ||--o{ AUDIT_LOG : "actor of"
    APP_USER ||--o{ EXPORT_LOG : "actor of"
    APP_USER ||--o{ QUERY_LOG : "actor of"

    CONTROL_POINT {
        bigint control_point_id PK
        varchar external_id
        double latitude
        double longitude
        double easting
        double northing
        varchar datum
        numeric accuracy
        enum source
        enum status
        geometry geom
        int village_id FK
    }
    UPLOAD_BATCH {
        bigint batch_id PK
        varchar filename
        int village_id FK
        int rows_accepted
        int rows_rejected
    }
    GEOREF_JOB {
        bigint georef_job_id PK
        varchar scope
        int village_id FK
        enum transform_type
        numeric rmse
        enum status
        numeric weight_dgps
        numeric weight_drone
        numeric weight_legacy
        text justification
        geometry before_geom
        geometry after_geom
    }
    FARMER_PARCEL_LINK {
        bigint id PK
        bigint parcel_id FK
        varchar farmer_id
        varchar role
```

```
    varchar consent_ref
}
CROP_SURVEY {
    bigint id PK
    bigint parcel_id FK
    varchar season
    varchar crop
    date sown_date
    varchar dcs_ref
}
APP_USER {
    uuid keycloak_id PK
    varchar username
    enum role
    int region_id FK
    int taluk_id FK
    int village_id FK
}
PERMISSION_MATRIX {
    bigint id PK
    enum role
    varchar resource
    boolean can_create
    boolean can_read
    boolean can_update
    boolean can_delete
    boolean can_export
}
AUDIT_LOG {
    bigint audit_id PK
    uuid actor
    varchar action
    varchar entity_type
    varchar entity_id
    geometry before_geom
    geometry after_geom
    timestamptz occurred_at
}
EXPORT_LOG {
    bigint export_id PK
    uuid actor
    varchar format
    geometry extent_geom
    int record_count
}
QUERY_LOG {
    bigint query_id PK
    uuid actor
    varchar query_type
    jsonb parameters
}
ANOMALY {
    bigint anomaly_id PK
```

```
        enum anomaly_type
        bigint parcel_id FK
        int village_id FK
        numeric variance_pct
        enum variance_band
        enum status
    }
    CERTIFIED_EXTRACT {
        bigint extract_id PK
        bigint parcel_id FK
        char ulpin
        uuid generated_by
        varchar digilocker_uri
        varchar status
    }
    PARCEL {
        bigint parcel_id PK
        char ulpin UK
    }
    VILLAGE {
        int village_id PK
        varchar name
    }
```

Part C — Data-Flow Diagrams

C.1 Level-1 Data Flow (the whole platform)

Processes (rounded), data stores (cylinders), and external entities (rectangles) and what flows between them.

```
flowchart TB
    citizen["Citizen"]
    officer["Officer"]
    field["Field Officer"]
    nila["Nilamagal / RoR"]
    agristack["AgriStack"]

    p1(["P1 Map Serving<br/>WMS/WFS/WMTS"])
    p2(["P2 Search & Query"])
    p3(["P3 Cadastral Editing"])
    p4(["P4 Mutation Sync"])
    p5(["P5 Georeferencing"])
    p6(["P6 Anomaly Analytics"])
    p7(["P7 AgriStack Integration"])
    p8(["P8 Auth & RBAC"])

    geo[("geo — parcels / admin")]
    rords[("ror — record of rights")]
    mutds[("mut — mutations / ULPIN")]
    grefds[("gref — control points / jobs")]
    audds[("aud — audit / export / query")]
    agrids[("agri — farmer / crop")]

    citizen --> p8
    officer --> p8
    field --> p8
    p8 --> p1 & p2 & p3 & p5 & p6

    p1 --> geo
    p2 --> geo
    p2 --> rords
    p2 --> audds
    p3 --> geo
    p3 --> audds
    p4 <--> nila
    p4 --> rords
    p4 --> mutds
    p4 --> geo
    field --> p5
    p5 --> grefds
    p5 --> geo
    p5 --> audds
    p6 --> geo
    p6 --> rords
    p6 --> audds
    p7 <--> agristack
    p7 --> agrids
    p7 --> geo
```

C.2 Bidirectional Mutation Data Flow (RoR <-> WebGIS)

The core "Online Mutation" loop: a change can start on either side, but nothing syncs back

until a DoSLR officer digitally approves it.

```
flowchart LR
    subgraph external["Nilamagal / RoR + Collabland"]
        rorsys["RoR textual record"]
        collab["Collabland spatial record"]
    end

    subgraph platform["WebGIS Platform"]
        cdc["Change-Data-Capture<br/>/ secure API listener"]
        draft["Draft mutation<br/>(geometry + attributes)"]
        approve{"DoSLR officer<br/>digital approval?"}
        commit["Commit to cadastral layer<br/>+ pgMemento version"]
        syncback["Sync back to<br/>Nilamagal + Collabland"]
        audit[("audit_log<br/>before/after")]
    end

    rorsys -->|"mutation event<br/>(subdiv/amalg/attr)"| cdc
    cdc --> draft
    draft --> approve
    approve -->|rejected| audit
    approve -->|approved| commit
    commit --> audit
    commit --> syncback
    syncback --> rorsys
    syncback --> collab
```

C.3 Georeferencing Data Flow (three input surfaces, one engine)

DGPS/GNSS points arrive from the mobile app, the web workbench, or a bulk upload — all land in the same control-point library, all feed one GDAL engine, all write to one audit trail.

```
flowchart TB
    mobile["Mobile app<br/>DGPS/GNSS via BT/NTRIP"]
    webman["Web manual entry<br/>+ drag-and-link"]
    upload["CSV/Excel bulk upload"]

    cplib[("control_point<br/>(single GCP library)")]
    review{"Officer review<br/>accept / reject / flag"}
    dryrun["GDAL dry-run transform<br/>affine / poly / spline / TPS"]
    rmse{"RMSE within<br/>tolerance?"}
    commitg["Commit bounded<br/>re-georeference"]
    parcel[("geo.parcel<br/>cadastral layer")]
    auditg[("audit_log + georef_job<br/>inputs, transform, RMSE")]

    mobile --> cplib
    webman --> cplib
    upload --> cplib
    cplib --> review
    review -->|accepted| dryrun
    review -->|rejected/flagged| auditg
    dryrun --> rmse
    rmse -->|no| review
    rmse -->|yes| commitg
    commitg --> parcel
    commitg --> auditg
```

Part D — Process Flowcharts

D.1 Online Mutation Workflow (end-to-end)

```
flowchart TD
    start(["Mutation triggered"]) --> origin{"Origin?"}
    origin -->|"From RoR (Nilamagal)"| ingest["Ingest event via CDC/API"]
    origin -->|"From WebGIS (officer draws)"| drawedit["Officer edits geometry<br/>(sp
    ingest --> draftm["Create draft mutation_event<br/>capture before/after"]
    drawedit --> draftm
    draftm --> ulpincalc["Compute ULPIN lifecycle<br/>retire parents / issue children"
    ulpincalc --> submit["Submit for approval"]
    submit --> review{"DoSLR officer<br/>approves?"}
    review -->|No| reject["Status = rejected<br/>log reason"]
    reject --> done1(["End"])
    review -->|Yes| commit["Status = approved<br/>commit to cadastral layer"]
    commit --> version["pgMemento writes version<br/>+ immutable audit_log"]
    version --> sync["Sync back to Nilamagal + Collabland"]
    sync --> synced["Status = synced"]
    synced --> done2(["End"])
```

D.2 Field Officer — Offline-First Mobile Workflow

```
flowchart TD
    login(["Officer logs in<br/>(Keycloak token)"]) --> download["Download assigned vi
    download --> field["At field site"]
    field --> connect["Pair DGPS/GNSS rover<br/>(Bluetooth / NTRIP)"]
    connect --> capture["Capture control point<br/>lat/long/easting/northing/accuracy"
    capture --> store["Save to local Room DB<br/>(works offline)"]
    store --> more{"More points?"}
    more -->|Yes| capture
    more -->|No| online{"Connectivity<br/>available?"}
    online -->|No| queue["WorkManager queues sync<br/>retry with backoff"]
    queue --> online
    online -->|Yes| push["Sync points to backend<br/>(source = mobile_dgps)"]
    push --> lib["Points enter control_point<br/>library, status = pending"]
    lib --> visible(["Visible to web reviewers<br/>in near real-time"])
```

D.3 Parcel Search + Role-Based Access Gating

The same search endpoint serves everyone, but what comes back depends on the role — owner PII never reaches a public or anonymous session.

```
flowchart TD
    q(["User submits parcel search"]) --> authn{"Authenticated?"}
    authn -->|No / anonymous| publictier["Public tier:<br/>geometry, Survey No., admin
    authn -->|Yes| role{"Role permits<br/>owner data?"}
    role -->|"Citizen (authenticated)"| limited["Limited attributes<br/>still no owner
    role -->|"Officer / Inspector / Surveyor"| full["Full RoR attributes<br/>owner, ter
    ownersearch{"Search was by<br/>owner name?"}
    publictier --> blockowner["Owner-name search blocked"]
    limited --> ownersearch
    full --> ownersearch
    ownersearch -->|Yes| logq["Log query to query_log<br/>(actor, params, count)"]
    ownersearch -->|No| result
    logq --> result(["Return results"])
    blockowner --> result
```

D.4 Authentication Flow (Keycloak)

```
flowchart TD
    start(["User opens portal / app"]) --> who{"Citizen or Officer?"}
    who -->|Citizen| cmethod{"Method?"}
    cmethod -->|DigiLocker| dl["OIDC broker to DigiLocker"]
    cmethod -->|Mobile OTP| otp["SMS OTP via Keycloak"]
    dl --> citizentoken["Issue citizen token<br/>(limited scope)"]
    otp --> citizentoken
    who -->|Officer| sso["SSO / username+password"]
    sso --> priv{"Privileged role?"}
    priv -->|Yes| mfa["Require MFA<br/>(TOTP / WebAuthn)"]
    priv -->|No| officertoken
```

```
    mfa --> officertoken["Issue officer token<br/>(role + jurisdiction scope)"]
    citizentoken --> guard["Every API call checks<br/>permission_matrix + scope"]
    officertoken --> guard
    guard --> done(["Access granted to allowed resources"])
```

D.5 Discrepancy & Anomaly Detection Pipeline

Runs as a scheduled job, persists findings to the `anomaly` table, and powers the officials' dashboard with colour-coded variance bands.

```
flowchart TD
    trigger(["Scheduled analytics run<br/>(Celery job)"]) --> loop["For each parcel in
    loop --> area["Compare ST_Area(geom)<br/>vs ror_extent_sqm"]
    area --> band{"Variance band?"}
    band -->|"<= 1%"| green["Green"]
    band -->|"1-5%"| amber["Amber"]
    band -->|"> 5%"| red["Red"]
    loop --> boundary["Overlay vector on orthomosaic<br/>flag boundary deviation"]
    loop --> checks["Run record-map checks:<br/>missing RoR, missing geometry,<br/>dup
    green --> persist[("Write to anomaly table<br/>type, band, highlight_geom")]
    amber --> persist
    red --> persist
    boundary --> persist
    checks --> persist
    persist --> dash["Dashboard: filter by<br/>region/taluk/village/band/type"]
    dash --> drill["Drill-down opens pre-zoomed map<br/>with issue highlighted"]
    drill --> export(["Export anomaly list PDF/Excel<br/>+ variance heat-map"])
```

D.6 Certified Extract Generation + DigiLocker Issuance

```
flowchart TD
    req(["Authorised user requests<br/>certified extract for a parcel"]) --> gather["Ga
    gather --> render["Render extract (mapfish-print)<br/>map + attribute block"]
    render --> sign["Apply digital signature"]
    sign --> record[("Write certified_extract row<br/>signature_ref, generated_by")]
    record --> push{"Issue channel?"}
    push -->|DigiLocker| dl["Push to DigiLocker<br/>store digilocker_uri"]
    push -->|Direct| download["Signed PDF download"]
    dl --> done(["Citizen retrieves from DigiLocker"])
    download --> done
```

E.1 Mutation Event — Status Lifecycle

```
stateDiagram-v2
    [*] --> draft: created
    draft --> submitted: submit for approval
    submitted --> approved: DoSLR officer approves
    submitted --> rejected: officer rejects
    approved --> synced: pushed to Nilamagal + Collabland
    rejected --> [*]
    synced --> [*]
```

E.2 Control Point — Status Lifecycle

```
stateDiagram-v2
    [*] --> pending: ingested (mobile / web / upload)
    pending --> accepted: officer accepts
    pending --> rejected: officer rejects
    pending --> flagged: officer flags for query
    flagged --> accepted: resolved
    flagged --> rejected: resolved
    accepted --> [*]: promoted to GCP library
    rejected --> [*]
```

E.3 Georeferencing Job — Status Lifecycle

```
stateDiagram-v2
    [*] --> dry_run: run transform preview
    dry_run --> committed: RMSE OK, officer commits
    dry_run --> failed: transform error
    committed --> reverted: officer reverts edit
    failed --> [*]
    reverted --> [*]
    committed --> [*]
```

E.4 Parcel & ULPIN — Lifecycle Through a Mutation

```
stateDiagram-v2
    direction LR
    [*] --> active: parcel created, ULPIN issued
    active --> retired: superseded by mutation
    note right of retired
        Sub-division: parent retired,
        children get new ULPINs.
        Amalgamation: parents retired,
        one new ULPIN issued.
        Lineage preserved in ulpin_lineage.
```

```
    end note
    retired --> [*]
    active --> active: attribute-only mutation (same ULPIN)
```

Part F — Sequence Diagrams

F.1 Field Georeferencing — End to End

```
sequenceDiagram
    actor FO as Field Officer
    participant M as Mobile App
    participant API as FastAPI Backend
    participant DB as PostGIS
    participant ENG as GDAL Engine
    actor OFFR as Web Officer

    FO->>M: pair rover, capture DGPS point
    M->>M: store in local Room DB (offline)
    M->>API: sync point (source=mobile_dgps)
    API->>DB: insert control_point (status=pending)
    OFFR->>API: review pending points
    API->>DB: update status=accepted
    OFFR->>API: request dry-run georeference (village scope)
    API->>ENG: run TPS transform with weighted GCPs
    ENG-->>API: transformed geometry + RMSE
    API-->>OFFR: preview + RMSE readout
    OFFR->>API: commit
    API->>DB: update parcels + georef_job + audit_log
    API-->>OFFR: committed (version saved)
```

F.2 Online Mutation with Sync-Back

```
sequenceDiagram
    participant RoR as Nilamagal / RoR
    participant API as FastAPI Backend
    participant DB as PostGIS
    actor OFFR as DoSLR Officer
    participant CL as Collabland

    RoR->>API: mutation event (sub-division)
    API->>DB: create mutation_event (status=draft)
    API->>DB: retire parent ULPIN, issue child ULPINs
    API->>OFFR: present for approval (before/after preview)
    OFFR->>API: approve
    API->>DB: commit geometry + attributes (status=approved)
    API->>DB: pgMemento version + immutable audit_log
    API->>RoR: sync updated textual record
    API->>CL: sync updated spatial record
```

```
    API->>DB: status=synced
    API-->>OFFR: confirmation
```

F.3 Citizen Parcel Search (Public Tier)

```
sequenceDiagram
    actor C as Citizen
    participant W as Web Portal
    participant KC as Keycloak
    participant API as FastAPI Backend
    participant GS as GeoServer
    participant DB as PostGIS

    C->>W: search by Survey No.
    W->>KC: anonymous / citizen token
    KC-->>W: token (public scope)
    W->>API: search request + token
    API->>API: enforce permission_matrix (no owner PII)
    API->>DB: query parcel geometry + public attributes
    DB-->>API: parcel result (no owner data)
    API-->>W: public-tier result
    W->>GS: request WMS tiles for extent
    GS-->>W: rendered map tiles
    W-->>C: parcel shown on map (public attributes only)
```

All geometry is stored in EPSG:4326 (WGS 84) per RFP Sec 18.2.3; display reprojection to India Zone II happens in the service layer. Row-level history, rollback and "as-of-date" queries are provided by pgMemento on top of these tables.